

Executive Summary

Overview

A security assessment was conducted on the provided test server to evaluate its resilience against fuzzing and Denial-of-Service (DoS) attacks. The goal was to identify vulnerabilities, exploit them, and subsequently mitigate the identified risks.

Key Findings

- The server was found to be vulnerable to buffer overflow attacks due to unsafe string handling.
- A fuzzing attack successfully caused unexpected behavior and crashes.
- A DoS attack overloaded the server, leading to service disruption.
- A revised version of the server was developed that successfully mitigates these vulnerabilities.

Recommendations

- Implement secure coding practices, such as input validation and proper buffer handling.
- Use safe string functions (e.g., `strncpy` instead of `strcpy`).
- Implement rate limiting to mitigate DoS risks.
- Conduct regular security audits to identify new vulnerabilities.

Technical Briefing

1. Fuzzing Attack

Methodology

- A Python script was developed to send increasingly large, random inputs to the server.
- The script monitored for crashes, unexpected responses, or connection terminations.

Results

- The server crashed when receiving an input exceeding 64 bytes due to a buffer overflow in `vulnerable_function()`.
- A payload of 80+ bytes reliably caused a segmentation fault.

2. DoS Attack

Methodology

- A Python script was developed to flood the server with multiple concurrent connections.
- 100 threads were used to send oversized payloads, leading to resource exhaustion.

Results

- The server stopped responding under high load.
- Logs indicated connection failures due to excessive socket requests.

3. Server Hardening

Identified Vulnerability

- The `strcpy()` function in `vulnerable_function()` allowed arbitrary data to overflow the buffer, potentially leading to remote code execution.

Fix Implemented

- Replaced `strcpy()` with `strncpy()` to prevent buffer overflow.
- Added null-termination to ensure data integrity.
- Restricted input size when reading from clients.

Validation

- The fuzzing script was rerun against the patched server, and no crashes occurred.
- The DoS script was executed, and the server remained stable under load.

Screenshot Evidence

(Insert screenshot showing stability of patched server)

Conclusion

The original server was vulnerable to both fuzzing and DoS attacks due to unsafe string handling and lack of rate limiting. By implementing proper buffer management and restricting input size, the revised server successfully mitigated these vulnerabilities. Organizations should enforce secure coding practices and conduct regular penetration tests to maintain a robust security posture.
